

P07 Settlement spec

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P06.

1. What to do, in one sentence

Write the full specification for how money is settled, payment methods, discount reasons, No Charge, On Hold accounts, platform payouts, the cash drawer and Day Close, into the contract and schema docs, with **no code at all**, so P08, P09 and P10 build from one agreed design.

2. Modules

M10 Payments, pulled forward with an adjusted scope.

M16 Settlement and Day Close, new.

M17 Delivery and Platform Orders, the payouts part.

3. Why this is its own prompt

These pieces touch each other everywhere.

A bill charged to an account changes what "paid" means. Day Close needs every payment's business date. A cash collection for an old account shows up in today's drawer. A closed day must refuse a payment correction.

If each build prompt designed its own slice, the slices would disagree. So the design is written once, here, and reviewed before any code.

`docs/CAFFEZA-BUILD-PLAN.md` section 7 already requires a module's spec to be committed before its code.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P07-settlement-spec.md`.
2. `git pull . git status` should show only that file.
3. `docs/prompts/README.md` must show P06 as Done. If not, stop and tell me.
4. No tests to run. This prompt changes no code. If you find yourself opening a `.js` file to change it, stop.

5. Files to read first

1. `docs/GLOSSARY.md`, all of it.
2. `docs/REPORT-SPEC.md` sections 1 and 3, especially R2, R5, R6, R7, R16 and R17.
3. `docs/RECONCILIATION-RULES.md`, all of it.
4. `docs/TEST-DATA.md` sections 2 and 4.
5. `docs/CAFFEZA-PROFILE.md` sections 10, 11 and 12.
6. `docs/CAFFEZA-BUILD-PLAN.md` section 6, the new audit actions.
7. `docs/API-CONTRACT.md`: section 14 bills, the M7 section, the M8 section, and the M17 section from P06.
8. `docs/DB-SCHEMA.md`: section 9 orders, section 12 bills, section 13 auditlogs, section 17 settings, section 19 stations.
9. Read, but do not change, `server/models/Bill.js`, `server/models/Order.js`, `server/services/billService.js` and `server/models/AuditLog.js`, so the spec fits the code that exists.

6. What to write, and where

Doc	What changes
<code>docs/API-CONTRACT.md</code>	New section "M10 Payments". New section "M16 Settlement and Day Close". Payouts added to the "M17" section. Section 14 points at M10 and M16 for the bill changes. The M7 section gains the new settings

Doc	What changes
	groups. The M8 section 1 table gains every new audit action.
docs/DB-SCHEMA.md	New sections 20 to 25, one per new collection. New fields in sections 9, 12 and 17. New enum values in section 13.
docs/GLOSSARY.md	The small changes in section 13 of this prompt
docs/REPORT-SPEC.md	The small changes in section 13 of this prompt
docs/RECONCILIATION-RULES.md	The small changes in section 13 of this prompt
docs/CAFFEZA-BUILD-PLAN.md	Prompt titles and audit actions, section 13 of this prompt

Write each endpoint in the same style as the existing contract: method and path, roles, request example, response example, every rule with its status code and error code, and a permission summary table per module.

Write each collection in the same style as the existing schema: a field table with type, required, links and notes, then indexes, then the reason for each index.

Sections 7 to 12 below are the design. Turn them into the docs. Where this prompt leaves a small detail open, choose what fits the existing code best, and list every such choice in your final summary so it can be checked.

7. Payment methods (M10)

7a. Collection `paymentmethods` , DB-SCHEMA section 20

Field	Type	Rules
<code>code</code>	String	Required. 2 to 20 characters, capital letters, digits and <code>_</code> , starting with a letter. Unique per restaurant. Never changes after creation.
<code>name</code>	String	Required. 1 to 30 characters. What staff see.
<code>kind</code>	String	<code>IN_HAND</code> or <code>PLATFORM</code> . Never changes after creation.
<code>orderTypes</code>	[String]	Which order types may use it. At least one of <code>DINE_IN</code> , <code>TAKEAWAY</code> , <code>DELIVERY</code> . Default all three.
<code>platformCode</code>	String or null	For a method that is a delivery platform's own payment, the code from <code>server/config/platforms.js</code> , like <code>SWIGGY</code> . Otherwise null.
<code>tallyLedgerCode</code>	String or null	Up to 20 characters. Caffeza's are in <code>docs/CAFFEZA-PROFILE.md</code> section 10.
<code>commissionBps</code>	Number or null	<code>PLATFORM</code> methods only. Integer 0 to 10000. Null means "rate not set".
<code>displayOrder</code>	Number	Integer, default 0
<code>isActive</code>	Boolean	Default true. Never deleted.

Index { `restaurantId`, `code` } unique.

Every restaurant always has four built-in methods, created if missing, by `code`, without ever overwriting an existing one:

`CASH` Cash, `CARD` Card, `UPI` UPI, all `IN_HAND` and active, and `OTHER` Other, `IN_HAND` , inactive.

These match the four values the old fixed list allowed, so every existing payment still points at a real method.

7b. Endpoints

Method and path	Roles	Notes
GET /payment-methods	All signed in	Active methods by <code>displayOrder</code> . ? <code>includeInactive=true</code> for OWNER and MANAGER.
POST /payment-methods	OWNER	Create
PATCH /payment-methods/:id	OWNER	Everything except <code>code</code> and <code>kind</code> . A commission change affects only payments taken after it.

7c. Taking a payment

`POST /bills/:billId/payments` keeps its shape. `method` now holds a method code instead of a value from the fixed list.

Rules, each 422 `PAYMENT_METHOD_NOT_ALLOWED` with a message naming the reason:

1. The code is an active method of this restaurant.
2. The method's `orderTypes` include the bill's order type.
3. On a delivery bill with a platform, only the method whose `platformCode` matches that platform. A Swiggy order is paid by the Swiggy method and nothing else.
4. A method with a `platformCode` is used only on delivery bills from that platform.

Each payment freezes, as new additive fields:

`methodName` , `methodKind` , `tallyLedgerCode` , `commissionBps` , null for `IN_HAND` ,
and `businessDate` , worked out from `receivedAt` with the restaurant's business day start, using `businessDateFor` .

The fixed enum on `payments[].method` in the model is removed, and the check moves into the service. That relaxes a rule rather than tightening one, so existing documents stay valid.

Old payments have null frozen fields. Readers treat a null `methodKind` as `IN_HAND`, and a null `businessDate` as the bill's `businessDate`.

7d. Correcting a payment's method

POST `/bills/:billId/payments/:paymentId/correct`

Roles: OWNER, MANAGER.

Body: `{ method, reason }`, `reason` 1 to 200 characters, required.

Only the method changes. The amount never does: to change an amount, void and bill again.

The new method must pass every rule in 7c.

The payment keeps a `corrections` array of `{ fromMethod, toMethod, by, at, reason }`, and its frozen method fields are replaced with the new method's.

Writes audit action `PAYMENT_METHOD_CORRECTED`, entity `BILL`, with the amount.

Refused with 409 `DAY_CLOSED` when the payment's business date is closed, once P10 exists.

7e. Discount reasons

The discount request changes the same way cancels did in P04:

`{ kind, valueInPaise | rateBps, reasonCode, note?, fundedBy? }`.

New file `server/config/discountReasons.js`, mirrored on the client:

Code	Label	Platform reason
<code>ZOMATO_GOLD</code>	Zomato Gold	yes
<code>DINEOUT</code>	Dineout	yes
<code>EAZYDINER</code>	EazyDiner	yes
<code>REGULAR_GUEST</code>	Regular guest	no
<code>REFERRAL</code>	Referral	no
<code>STAFF_OFFICE</code>	Staff or office	no
<code>MERCHANT_PROMO</code>	Merchant promo	no

Code	Label	Platform reason
SERVICE_RECOVERY	Service recovery	no
OTHER	Other, note required	no

`fundedBy` is `RESTAURANT` or `PLATFORM`, default `RESTAURANT`. `PLATFORM` is allowed only with a platform reason.

The bill's `discount` gains `reasonCode` and `fundedBy`. The existing `reason` text holds the note.

Who may discount: OWNER and MANAGER, as today.

Plus CASHIER, for platform reasons only, when a new setting

`settings.discounts.cashierMayApplyPlatformDiscounts` is true. Default false. `TO CONFIRM` with Caffeza, who apply these at the till.

8. No Charge (M16)

No Charge closes an order without a bill. It is not a sale and takes no invoice number.

8a. The data

`orders.status` gains `NO_CHARGE`. Append only.

Orders gain `noCharge`, null unless the status is `NO_CHARGE`:

`{ reasonCode, note, approvedBy, at, businessDate, valueInPaise }`.

`valueInPaise` is the sum of the order's live line totals, at menu price, before GST, frozen at that moment.

New file `server/config/noChargeReasons.js`, mirrored on the client:

`CORPORATE_OFFICE` Corporate office order, `STAFF_MEAL` Staff meal, `OWNER_GUEST` Owner's guest, `TASTING` Tasting or trial, `SERVICE_RECOVERY` Service recovery, `OTHER` Other, note required.

8b. The endpoint

POST `/orders/:orderId/no-charge`

Roles: OWNER, MANAGER.

Body: `{ version, reasonCode, note? }`.

Rules, each 422 BUSINESS_RULE_VIOLATED :

1. The order is OPEN or READY_TO_BILL .
2. It has no live bill. Message: "Void the bill first."
3. It has no line still waiting to be sent to the kitchen. Message: "Send or cancel the unsent items first."
4. It has at least one live line.

Effect, in one transaction: status NO_CHARGE , the table freed, noCharge filled in, and audit action NO_CHARGE_GIVEN , entity ORDER , with amountInPaise equal to valueInPaise .

Stock was already deducted when the lines fired, so nothing else moves.

9. On Hold accounts (M16)

An On Hold bill is a sale whose money arrives later, from a named account such as "W-330 Office".

9a. Collection accounts , DB-SCHEMA section 21

Field	Type	Rules
name	String	1 to 40 characters, unique per restaurant ignoring case, through nameLower
contactName , phone , note	String or null	Optional
openingBalanceInPaise	Number	Integer 0 or more. Set when the account is created, never edited. It is also written as an OPENING entry.
isActive	Boolean	Default true

9b. Collection accountentries , DB-SCHEMA section 22

The account's ledger. Entries are never edited or deleted.

Field	Type	Rules
accountId	ObjectId	Required
type	String	OPENING , CHARGE , CHARGE_REVERSED , COLLECTION , ADJUSTMENT
direction	String	UP or DOWN . Fixed by type, except ADJUSTMENT , which takes either. OPENING and CHARGE are UP . CHARGE_REVERSED and COLLECTION are DOWN .
amountInPaise	Number	Integer above 0. Always positive. The direction carries the sign.
billId , billNumber		For CHARGE and CHARGE_REVERSED
method , methodName , methodKind		For COLLECTION , frozen like a payment
reference , note	String or null	
businessDate	String	When the entry happened, by the business day
at , by		UTC time and user

Outstanding balance is the UP amounts minus the DOWN amounts.

9c. Bills

bills.status gains ON_ACCOUNT . Append only.

Bills gain account: { accountId, accountName } ,

chargedToAccountInPaise , chargedAt , chargedBy , all null unless charged.

9d. Endpoints

Method and path	Roles	Notes
GET /accounts	OWNER, MANAGER, CASHIER	Each with its outstanding balance and the date of its oldest uncollected charge
POST /accounts	OWNER, MANAGER	Create, with an optional opening balance
PATCH /accounts/:id	OWNER, MANAGER	Name, contact, note, active. Never the opening balance.
POST /bills/:billId/charge-to-account	OWNER, MANAGER	Body { accountId }. Who else may do this: TO CONFIRM with Caffeza.
POST /accounts/:id/collections	OWNER, MANAGER, CASHIER	Body { method, amountInPaise, reference?, note? }. IN_HAND methods only.
POST /accounts/:id/adjustments	OWNER	Body { direction, amountInPaise, reason }. Writes audit ACCOUNT_BALANCE_ADJUSTED, entity ACCOUNT.
GET /accounts/:id/statement?from&to	OWNER, MANAGER	Balance at the start, every entry with a running balance, balance at the end

Charging a bill:

1. The bill is UNPAID and not voided, and the account is active.
2. The amount charged is the bill total minus what was already paid, and it must be above zero.
3. In one transaction: the bill becomes ON_ACCOUNT, the order becomes BILLED and frees its table exactly as a payment does, a CHARGE entry is written, and audit BILL_CHARGED_TO_ACCOUNT, entity BILL.

Voiding an ON_ACCOUNT bill is allowed, by the roles that void today, and writes a CHARGE_REVERSED entry in the same transaction.

A collection larger than the outstanding balance is refused with 422

`ACCOUNT_BALANCE_EXCEEDED` .

A collection counts in the drawer, or the bank, on the business date it arrives. It is never a sale.

10. Platform payouts (M17)

Platforms pay out in batches, usually weekly, covering a range of days.

10a. Collection `platformpayouts` , DB-SCHEMA section 23

Field	Type	Rules
<code>method</code> , <code>methodName</code>	String	A PLATFORM payment method's code, and its frozen name
<code>periodFrom</code> , <code>periodTo</code>	String	Business dates, inclusive
<code>amountReceivedInPaise</code>	Number	Integer 0 or more
<code>receivedOn</code>	String	The date it reached the bank, YYYY-MM-DD
<code>reference</code> , <code>note</code>	String or null	
<code>recordedBy</code> , <code>recordedAt</code>		
<code>isVoided</code> , <code>voidedBy</code> , <code>voidedAt</code> , <code>voidReason</code>		A wrong entry is voided, never edited

10b. Endpoints

Method and path	Roles
GET <code>/platform-payouts?</code>	OWNER, MANAGER

Method and path	Roles
method&from&to	
POST /platform-payouts	OWNER, MANAGER. Writes audit PLATFORM_PAYOUT_RECORDED , entity PAYOUT .
POST /platform-payouts/:id/void	OWNER. Body { reason } .

Two live payouts for the same method may not cover the same business date: 409 PAYOUT_PERIOD_OVERLAP .

10c. Expected payout

For one payout: every payment with that method whose businessDate is inside the period, each multiplied by $(10000 - \text{its frozen commissionBps}) / 10000$ through applyBasisPoints , then added up. Payments with no commission rate are listed separately as "rate not set" and left out of the expected figure. Nothing is invented.

11. Cash drawer (M16)

11a. Collection cashmovements , DB-SCHEMA section 24

Field	Type	Rules
type	String	OPENING_FLOAT , PAID_IN , PAID_OUT
amountInPaise	Number	Integer above 0
reason	String or null	1 to 200 characters. Required for PAID_IN and PAID_OUT .
businessDate	String	Always the current business date when entered. No back-dating.

Field	Type	Rules
at , by		
isVoided , voidedBy , voidedAt , voidReason		

One live opening float per business date: a partial unique index on { restaurantId, branchId, businessDate, type } where type is OPENING_FLOAT and isVoided is false.

11b. Endpoints

Method and path	Roles
GET /cash-movements? date	OWNER, MANAGER, CASHIER
POST /cash-movements	OPENING_FLOAT and PAID_IN : OWNER, MANAGER, CASHIER. PAID_OUT : OWNER, MANAGER, and writes audit CASH_PAID_OUT , entity CASH .
POST /cash-movements/:id/void	OWNER, MANAGER. Body { reason } .

12. Day Close (M16)

12a. The figures

A new service, `server/services/dayFiguresService.js` , with `computeDayFigures(req, businessDate, { session })` .

It returns the figures of `docs/REPORT-SPEC.md` R2, sections A to H, for one business date.

It is **the only place** day-level figures are computed. Day Close uses it now.

The R2 report in M19 will use the same function, so the screen and the stored snapshot can never disagree.

12b. Checks at close

A new service, `server/services/reconciliationService.js`. P10 builds checks C1, C3, C4, C6, C8 and C9 for one business date, which is what Day Close needs. P14 adds the rest and the range versions.

12c. Collection `dayclosures` , DB-SCHEMA section 25

One document per business date, unique on `{ restaurantId, branchId, businessDate }`.

Field	Type	Notes
<code>businessDate</code>	String	
<code>status</code>	String	<code>CLOSED</code> or <code>REOPENED</code>
<code>countedCashInPaise</code> , <code>expectedCashInPaise</code> , <code>differenceInPaise</code>	Number	From the latest close
<code>note</code>	String or null	Required when the cash difference is not zero
<code>snapshot</code>	Object	The full output of <code>computeDayFigures</code> at the latest close
<code>checks</code>	Array	The check results at the latest close
<code>closedBy</code> , <code>closedAt</code>		Latest close
<code>history</code>	Array	Every close and reopen: { action, by, at, note, countedCashInPaise, expectedCashInPaise, differenceInPaise }

12d. Endpoints

Method and path	Roles	Notes
POST /day-close	OWNER, MANAGER	Body { businessDate, countedCashInPaise, note? }
GET /day-close/:businessDate	OWNER, MANAGER	The blind count rule below applies
GET /day-close?from&to	OWNER, MANAGER	One row per date
GET /day-close/:businessDate/print?width=32	OWNER, MANAGER	The Day Close as plain text for a thermal printer, laid out on the server like the receipt
POST /day-close/:businessDate/reopen	OWNER	Body { reason } . Writes audit DAY_REOPENED , entity DAY .

Closing, POST /day-close :

1. The date is today's business date or earlier, and not already CLOSED .
2. Blockers, all reported together in one 422 DAY_NOT_READY with a list: an order opened on that business date still OPEN or READY_TO_BILL , a bill of that date still UNPAID , or any failed ERROR check from 12b.
3. A cash difference other than zero needs a note .
4. Effect, in one transaction: compute the figures, store the snapshot and checks, set CLOSED , add to history, and write audit DAY_CLOSED , entity DAY , with the day's bill total.

The blind count.

A MANAGER enters the counted cash without seeing the expected figure. Responses to a MANAGER leave out expectedCashInPaise and differenceInPaise , unless a new setting settings.dayClose.showCashDifferenceToManager is true. Default false. OWNER always sees both.

12e. The lock

Once a business date is `CLOSED`, every write that would change that date's figures is refused with 409 `DAY_CLOSED`, message "{date} is closed. An owner can reopen it."

One helper, `assertDayOpen(req, businessDate, { session })`, used by: creating a bill whose business date is closed, discounting, voiding, paying, correcting a payment, or charging to an account, on a bill of a closed date, any payment, No Charge, cash movement, account collection or payout, whose own business date is closed. P10 wires it into every one of these.

13. Smaller doc changes

Audit actions. In `docs/API-CONTRACT.md` M8 section 1, `docs/DB-SCHEMA.md` section 13, and `docs/CAFFEZA-BUILD-PLAN.md` section 6, list every new action and entity type:

actions `PAYMENT_METHOD_CORRECTED`, `NO_CHARGE_GIVEN`,
`BILL_CHARGED_TO_ACCOUNT`, `ACCOUNT_BALANCE_ADJUSTED`,
`PLATFORM_PAYOUT_RECORDED`, `CASH_PAID_OUT`, `DAY_CLOSED`,
`DAY_REOPENED`;

entity types `ACCOUNT`, `CASH`, `DAY`, `PAYOUT`.

In the build plan, `PLATFORM_PAYOUT_RECORDED` now has entity `PAYOUT`, written from P09.

Error codes. Add to `docs/CONVENTIONS.md` section 3:

`PAYMENT_METHOD_NOT_ALLOWED`, `ACCOUNT_BALANCE_EXCEEDED`,
`PAYOUT_PERIOD_OVERLAP`, `DAY_NOT_READY`, `DAY_CLOSED`.

Settings. In the M7 section and DB-SCHEMA section 17: groups `discounts` and `dayClose`, each with its one field and default.

Glossary.

Section 5: On Hold bill means a bill with status `ON_ACCOUNT`. No Charge order means an order with status `NO_CHARGE`.

Section 6: payment method kinds are `IN_HAND` and `PLATFORM`.

Section 7: payouts arrive as batches covering a range of business dates. Expected payout is worked out per batch, as in section 10c.

Report spec.

R2: computed by `computeDayFigures` .

R6: one row per payout batch, with expected, received and difference, plus a list of platform payments not yet covered by any payout.

R15: for lines cancelled by a whole-order cancel, the reason is the order's `cancelReasonCode` , as P04 noted.

R16: reads orders with status `NO_CHARGE` .

R17: reads `accounts` and `accountentries` .

Section 6, the fields table: replace the rows for P08, P09 and P10 with the real field names from this prompt.

Reconciliation rules.

C3: the equation reads payments plus `chargedToAccountInPaise` plus unpaid.

C9: cash collections come from `accountentries` of type `COLLECTION` with a cash method.

C10: from `accountentries` .

C11: per payout batch, as in section 10c.

C12: compares against `dayclosures.snapshot` .

Section 1: checks are built in `reconciliationService.js` , started in P10 and completed in P14.

Build plan, section 3.

Rename P09 to "On Hold accounts and platform payouts".

P08: "Payment methods, discount reasons and No Charge".

14. Check your own work

Before committing, read the finished docs as if you were about to build P08 from them, and answer in your summary:

1. Can every number in `docs/TEST-DATA.md` section 4, R2 sections B, D and G, be produced from the fields this spec defines? Name the field each one comes from.
2. Does every write in section 12e have exactly one place that calls `assertDayOpen` ?
3. Is there any endpoint without a permission row, or any collection without an index that starts with `restaurantId` ?

Fix any gap you find before committing.

15. Docs to update in PROJECT-STATE

1. The date line.
2. "Current stage": "Next: P08, payment methods, discount reasons and No Charge."
3. Module status: M10 and M16 become IN PROGRESS, "Specified in P07."
4. Decision log, dated today, one row each:

"Payment methods are a configured list per restaurant, each `IN_HAND` or `PLATFORM`, with a Tally code and an optional commission. Every payment freezes its method's name, kind, commission and business date. | Caffeza takes eight methods, and reports and payouts must not change when a method is renamed."

"No Charge is an order status, `NO_CHARGE`, with no bill and no invoice number. | It is not a sale, and must not touch the GST invoice series."

"On Hold is a bill status, `ON_ACCOUNT`, backed by an account ledger in `accountentries`. Collections are dated when the money arrives. | The sale happened on the day of the bill. The cash arrives on another day."

"Platform payouts are recorded as batches over a date range, and expected payout uses each payment's frozen commission. | That is how the platforms actually pay."

"Day figures are computed in one place, `computeDayFigures`, used by Day Close and by the R2 report. A closed day refuses every write that would change it, through one helper, `assertDayOpen`. | The printed close and the report must never disagree, and a closed day must stay closed."

"The Day Close cash count is blind for managers by default. | Standard practice against cash going missing."
5. Open questions: add "May a cashier charge a bill to an On Hold account?" and "May a cashier apply platform discounts? A setting exists, default off."
6. "What changed recently": a P07 entry at the top, and the oldest moved to the archive.
7. `docs/prompts/README.md`: mark P07 as Done, and rename P08 and P09 to match section 13.

16. Out of scope

Any code, any test, any model change. Specs only.

UPI QR codes on the bill.

Online payment gateways.

Automatic import of platform settlement reports.

17. Done when

1. Every doc in sections 6, 13 and 15 is updated.
2. The three questions in section 14 are answered, and every gap is fixed.
3. `npm run lint` still passes, as a check that no code was touched.
4. One commit on `main`: `add settlement spec for m10 m16 and payouts`.
5. Push `main`.
6. Print a short summary: the sections added to each doc, the answers to section 14, and every small detail you had to choose yourself.